# Task Report

Fidel Mbambo

1. Interpretation of the Problem

   The main goal of this project was to write a command-line script that creates a frequency allocation plan for a network of 19 cell towers (Cells A through S). We are given 6 distinct frequencies (110, 111, 112, 113, 114, 115).

   Because cell towers that are close to each other can cause signal interference, towers that are geographically near one another should not be given the same frequency. However, since there are only 6 frequencies and 19 towers, frequencies must be reused. The trick is making sure that the reuse only happens between towers that are far enough apart, so they don't interfere with each other. Finally, the program needs to output the allocations and generate a visual diagram showing the cell locations, connections, and assigned frequencies.

2. Chosen Approach and Algorithm

   I decided to implement the application using Node.js (JavaScript) due to its lightness and capability to output an SVG file effortlessly without employing heavyweight graphics libraries. Proximity Calculation (Distance Metric)For distance calculation, I decided to use Easting and Northing instead of Latitude and Longitude. Why: Easting and Northing values are already in meters (using a Cartesian coordinate system), which made it easier to use the Pythagorean formula for the calculations. On the other hand, the use of latitude and longitude would require coordinate conversion using the Haversine formula, which is complex and unnecessary for such a small area.

   **Allocation Algorithm**
   I used a **greedy distance-threshold approach**:

   - I set a distance threshold of **600 meters**. If two cells are closer than this, they are considered neighbours that can't share a frequency.
   - The program loops through each cell one by one.
   - For each cell, it checks what frequencies its already-allocated neighbours are using.
   - It picks the first available frequency that isn't being used nearby.
   - If all 6 frequencies happen to be blocked by nearby cells, it falls back to reusing a frequency from the furthest possible cell to keep interference minimal.

3. Alternatives I Considered

   Simulated Annealing / Graph Colouring (DSATUR): I was first interested in doing some work with the metaheuristic optimization techniques and graph colouring. Although those are neat things, those were a bit too much for just nineteen nodes. My basic greedy loop runs fast, is easy to debug, and yields a proper layout.

   Python with NetworkX & Matplotlib: Python is good for this type of computation, but I went with JavaScript as I wanted to implement my own SVG generation without having to worry about virtual environments or missing pip packages.

4. Assumptions Made
   - I presumed that the threshold value of six hundred meters represents a safe margin for interference between the two cells when operating at the same frequency.

   - I presumed that the input dataset would always return a valid pair of Easting and Northing coordinates corresponding to each cell ID.

   - I presumed that the command line tool which produces the text output in the console as well as the SVG file in the directory meets all requirements.

5. How I Determined Whether the Allocation Was Efficient

   To validate whether the assignment worked efficiently, the following was considered:

   - Console Output: I ran the code to ensure all the cells have an assignment of frequency ranging from 110 to 115 without any crashes.

   - Visual Inspection: I loaded the generated simple_frequency_map.svg file. The grey dashed lines are the representation of those cells that fall within the 600-meter limit. The assignment is validated by visual inspection of the colours of the connected nodes since connected neighbouring cells do not share the same colour (frequency).

6. Limitations of Your Solution

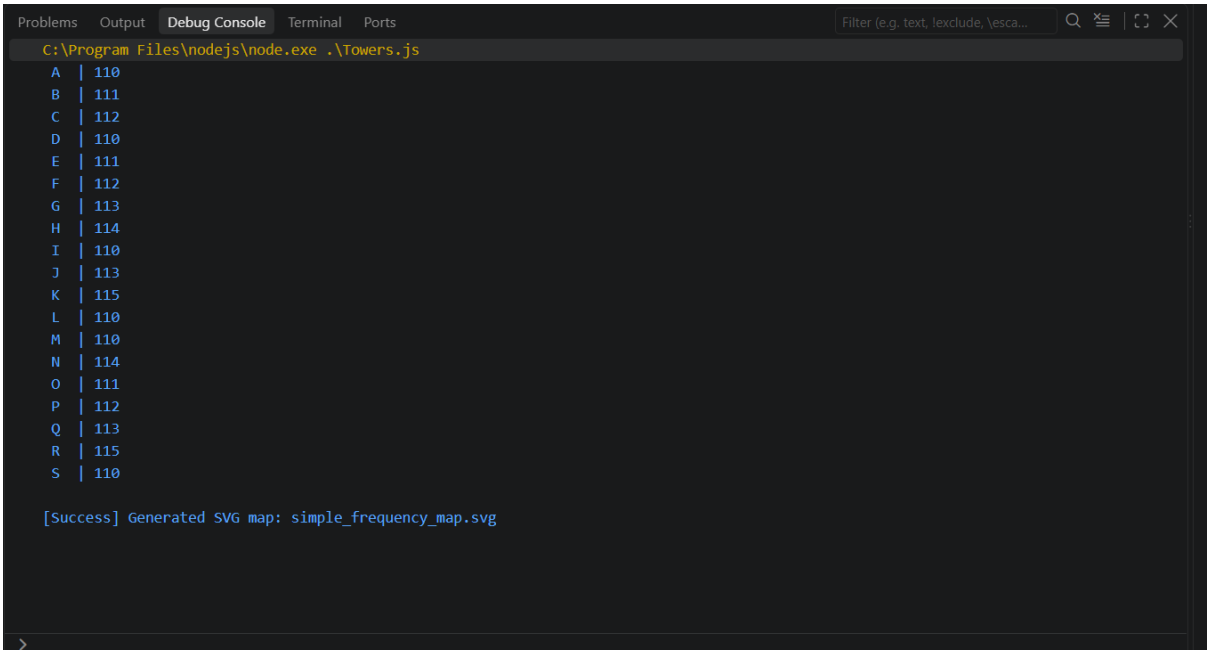
- Greedy Trap: Due to the sequential assignment of frequency values and the non-revisiting property, the algorithm may find itself trapped in a tricky situation when faced with more complicated network topologies.
- Fixed Threshold: 600 meters is an absolute constant in this algorithm. In practical mobile networks, radio interference is significantly dependent on the presence of hills, buildings, and antenna strength, and not only on the distance.

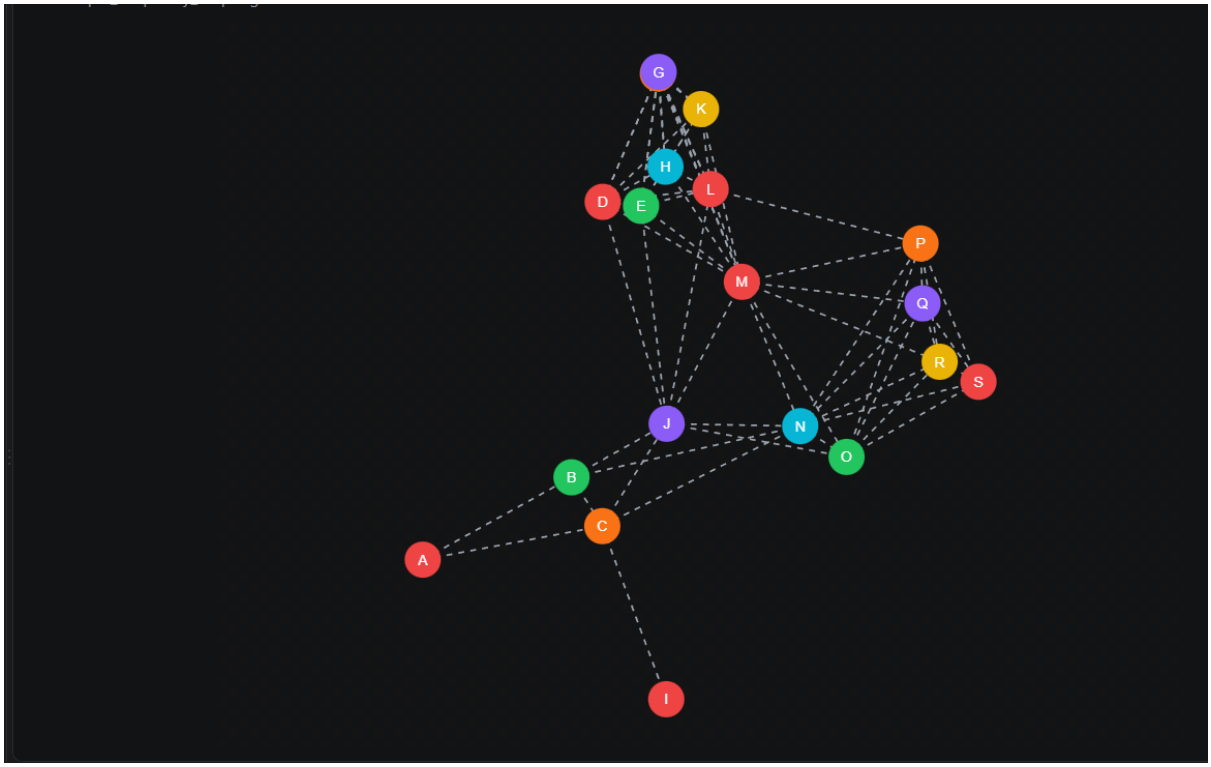
7. How Your Solution Could Be Improved for a Much Larger Network

   Spatial Indexing: Simple nested loops would become inefficient after a certain point. We should make use of some spatial indexing techniques such as R-Trees and Quadtrees to identify neighbouring cells efficiently.

   Global Optimization: Greedy method would not always give the optimal solution when dealing with a large number of cells in a network. We should move to some other algorithms such as Integer Linear Programming (ILP) (Google OR-Tools) or some kind of heuristic search techniques.

8. Visualisation






9. AI Usage Statements :

   AI tools used: ChatGPT

   AI was utilized to be an educational research assistant during the preparation stage to comprehend domain concepts and formatting. In particular, I used AI to:

   Comprehend the concepts of telecommunications, i.e., frequency reuse and co-channel interference for cell towers.

   Comprehend why geographic coordinate systems use Easting/Northing projected grid units for Euclidean straight-line distance calculation rather than Lat/Long.

   Get insight into various approaches to programmatic output generation, namely, how to use SVG viewports and how to translate geographic coordinates into SVG canvas coordinates without using any external libraries.

   Prompts & topics: The prompts were solely related to research questions, e.g.:

   "Describe the process of frequency reuse and co-channel interference for cellular towers."
   "How can I transform dynamic 2D coordinates into a fixed-size SVG viewBox manually?"
   "How can I format terminal output and SVG visualizations in Node.js without external libraries?"